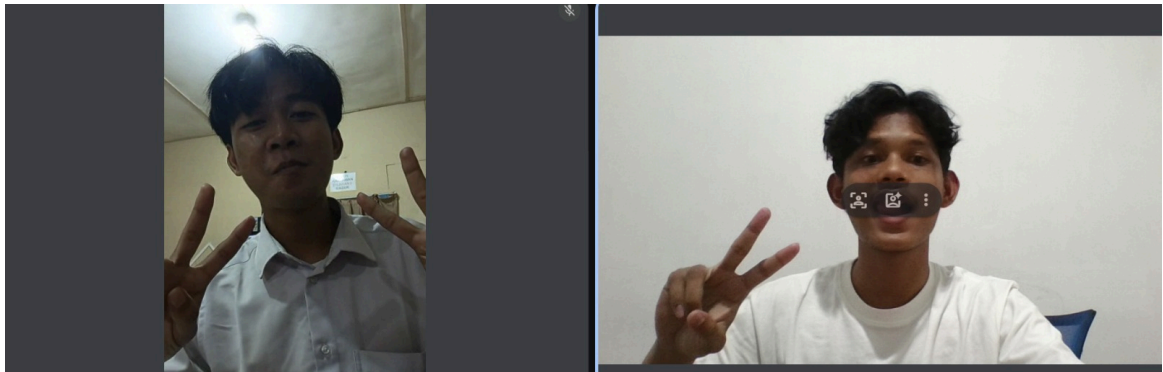


LAPORAN TUGAS BESAR

IF25-21013 Strategi Algoritma

ROBOCODE TANK ROYALE

Implementasi Algoritma Greedy



Disusun Oleh:

M. Ikhrom Azhar Al Muizzu-124140023
Febrian Yoel Anggara Saputra-124140031

Program Studi Teknik Informatika
Institut Teknologi Sumatera
2026

BAB 1: Deskripsi Tugas

Robocode Tank Royale merupakan permainan pemrograman berbasis strategi yang memungkinkan pemain membuat bot tank secara virtual untuk bertarung secara otomatis di dalam arena. Pada permainan ini, pemain tidak mengendalikan tank secara langsung, melainkan membuat program yang mengatur bagaimana bot bergerak, mendeteksi musuh, menghindari serangan, serta menembakkan peluru untuk memperoleh skor tertinggi.

Dalam permainan Robocode Tank Royale, setiap bot memiliki kemampuan untuk bergerak, memindai lawan menggunakan radar, serta menyerang menggunakan meriam. Bot juga harus memperhatikan berbagai aspek permainan seperti energi, panas meriam, kecepatan gerak, dan posisi musuh agar dapat bertahan hingga akhir pertandingan.

Pada tugas besar mata kuliah Strategi Algoritma ini, mahasiswa diminta untuk mengimplementasikan algoritma greedy dalam pembuatan bot Robocode Tank Royale. Strategi greedy digunakan sebagai dasar pengambilan keputusan bot dalam memilih aksi terbaik pada setiap kondisi permainan dengan tujuan memperoleh skor setinggi mungkin.

1.2 Deskripsi Permasalahan

Permasalahan utama pada tugas besar ini adalah bagaimana merancang sebuah bot yang mampu mengambil keputusan secara efektif di tengah kondisi permainan yang dinamis. Bot harus mampu menentukan tindakan terbaik berdasarkan kondisi arena, posisi musuh, energi yang dimiliki, serta peluang untuk memperoleh skor.

Bot yang dibuat harus mampu:

1. Bergerak menghindari serangan musuh dan dinding arena.
2. Memindai posisi musuh menggunakan radar.
3. Menentukan kapan harus menyerang atau menghindar.
4. Mengatur penggunaan energi agar tidak cepat habis.
5. Memilih target dan kekuatan tembakan yang optimal.

Setiap keputusan yang diambil bot harus menggunakan pendekatan greedy, yaitu memilih tindakan yang dianggap paling menguntungkan pada saat itu berdasarkan heuristic tertentu.

1.3 Tujuan Tugas

Tujuan dari tugas besar ini adalah:

1. Mengimplementasikan algoritma greedy pada permainan Robocode Tank Royale.
2. Membuat bot yang mampu memperoleh skor tinggi dalam pertandingan.
3. Mengeksplorasi beberapa heuristic greedy yang berbeda.
4. Menganalisis efektivitas strategi greedy yang digunakan pada bot.
5. Membandingkan performa beberapa alternatif strategi greedy yang telah dibuat.

1.4 Spesifikasi Tugas

Spesifikasi tugas besar yang harus dipenuhi adalah sebagai berikut:

1. Membuat 4 bot menggunakan bahasa C# (.NET), yang terdiri dari 1 bot utama dan 3 bot alternatif.
2. Setiap bot harus menggunakan strategi greedy dengan heuristic yang berbeda.
3. Bot harus dapat dijalankan pada game engine Robocode Tank Royale yang telah dimodifikasi oleh asisten.
4. Strategi greedy yang digunakan harus dijelaskan secara eksplisit pada laporan.
5. Bot utama akan dipertandingkan dengan bot dari kelompok lain pada kompetisi tugas besar.
6. Program harus memiliki struktur kode yang jelas dan mudah dipahami.
7. Repository proyek harus disusun sesuai struktur yang telah ditentukan pada spesifikasi tugas.

1.5 Output yang Diharapkan

Output yang diharapkan dari tugas besar ini adalah:

1. Bot Robocode Tank Royale yang dapat berjalan dengan baik pada game engine.
2. Implementasi algoritma greedy yang mampu mengambil keputusan secara efektif.
3. Bot yang mampu memperoleh skor tinggi dan bertahan selama pertandingan.
4. Laporan yang menjelaskan strategi greedy, implementasi program, serta hasil pengujian bot.

BAB 2: Landasan Teori

2.1 Dasar Teori Algoritma Greedy

Algoritma Greedy adalah algoritma pemecahan masalah yang bekerja dengan membuat keputusan yang tampak paling optimal pada setiap langkah tanpa mempertimbangkan konsekuensi dari keputusan tersebut terhadap langkah-langkah berikutnya. Kata "greedy" sendiri berarti serakah, yang mencerminkan sifat algoritma ini yang selalu mengambil pilihan terbaik yang tersedia saat itu tanpa melihat gambaran keseluruhan dari permasalahan.

Karakteristik utama algoritma Greedy:

1. Greedy Choice Property: Pilihan optimal lokal mengarah ke solusi global optimal.
2. Optimal Substructure: Solusi optimal dari masalah mengandung solusi optimal dari sub-masalah.
3. Efisiensi waktu: Umumnya lebih cepat dibanding *Dynamic Programming*.

2.2 Cara Kerja Program Secara Umum

Program bot Robocode Tank Royale bekerja dengan siklus *event-driven*. Bot menerima berbagai event dari game engine dan meresponsnya dengan aksi yang ditentukan oleh algoritma greedy yang diimplementasikan.

2.2.1 Bagaimana Bot Melakukan Aksi

Bot berinteraksi dengan lingkungan melalui serangkaian event dan aksi yang diproses setiap tick game. Pada setiap tick, bot menerima informasi tentang posisi dirinya, kondisi musuh, dan keadaan arena, kemudian menentukan aksi berdasarkan strategi greedy yang telah dirancang.

Mekanisme utama bot berjalan melalui event OnScannedBot, yang dipicu setiap kali radar mendeteksi musuh. Bot menerapkan sistem seleksi target berbasis skor yang mempertimbangkan empat faktor: jarak ke musuh, energi musuh yang tersisa, akurasi arah gun, dan kecepatan gerak musuh. Ketika radar mendeteksi musuh baru, bot menghitung skor target tersebut dan membandingkannya dengan target yang sedang dikunci. Bot akan mengalihkan kunci ke musuh baru hanya jika skor musuh baru lebih tinggi secara signifikan, target lama sudah lama tidak terdeteksi, atau musuh tersebut memang merupakan target yang sedang dikunci. Dengan mekanisme ini, bot mempertahankan kunci pada satu target hingga target tersebut dieliminasi, namun tetap responsif apabila ada musuh lain yang lebih mengancam melintas di depan radar.

Setelah target dikunci, bot secara bersamaan mengeksekusi tiga aksi: mengarahkan badan tegak lurus terhadap target untuk melakukan manuver strafe, mengunci radar pada posisi target dengan faktor pengali 1.5 untuk memastikan target selalu berada dalam jangkauan scan, serta mengarahkan gun ke posisi target. Selain itu, bot menerapkan mekanisme kontrol jarak terhadap target. Ketika jarak bot ke target kurang dari 180 unit, bot mundur untuk menjaga ruang gerak dan menghindari tabrakan. Sebaliknya, ketika jarak melebihi

500 unit, bot tidak lagi aktif mengejar melainkan berhenti bergerak dan tetap mempertahankan kunci radar pada target dari posisinya saat itu. Strategi ini memungkinkan bot untuk menembak dari jarak aman tanpa perlu mengorbankan posisi yang sudah menguntungkan, sekaligus menghemat energi yang akan terbuang sia-sia jika bot terus bergerak mengejar target yang jauh. Penembakan dilakukan secara oportunistik

Bot juga merespons event lain untuk bertahan hidup. Ketika terkena peluru (OnHitByBullet), bot membalik arah gerak dan bermanuver keluar dari lintasan tembak musuh. Ketika bertabrakan dengan bot lain (OnHitBot), bot mundur dan memanfaatkan momen tersebut untuk menembak jika musuh dalam kondisi lemah. Ketika menabrak dinding (OnHitWall), bot membalik arah gerak dan arah radar untuk kembali ke posisi yang lebih strategis.

2.2.2 Implementasi Algoritma Greedy ke dalam Bot

Fungsi objektif utama bot terdapat pada metode CalculateTargetScore, yang menghitung skor setiap musuh yang terdeteksi radar. Skor tersebut dihitung berdasarkan empat komponen: skor jarak yang memberikan nilai lebih tinggi kepada musuh yang lebih dekat dengan bobot 45%, skor kill yang memberikan nilai lebih tinggi kepada musuh yang energinya sudah rendah dengan bobot 35%, skor akurasi arah gun dengan bobot 15%, serta penalti yang mengurangi skor apabila musuh bergerak terlalu cepat atau berada terlalu dekat dalam radius berbahaya. Dengan fungsi ini, bot secara greedy selalu mengunci musuh yang paling "menguntungkan" untuk dieliminasi pada saat itu (musuh yang dekat, hampir mati, dan mudah ditembak).

Keputusan greedy kedua terdapat pada metode ChooseFirePower, di mana bot menentukan daya tembak secara lokal optimal berdasarkan kondisi saat itu tanpa mempertimbangkan konsekuensi jangka panjang. Bot memilih daya tembak tertinggi yang masih rasional: 3.0 untuk jarak di bawah 150 unit, 2.2 untuk jarak 150–350 unit, 1.4 untuk jarak 350–650 unit, dan 0.8 untuk jarak lebih dari 650 unit. Daya tembak kemudian dikurangi apabila energi musuh sudah sangat rendah cukup untuk menghabiskan musuh tanpa membuang energi berlebih atau apabila energi bot sendiri sudah kritis di bawah 20 unit sehingga bot tidak menghabiskan energi untuk menembak ketika kondisinya sedang terancam.

Keputusan greedy ketiga terjadi pada pemilihan kapan bot menembak. Bot tidak menembak setiap saat, melainkan hanya ketika dua kondisi terpenuhi secara bersamaan: gun sudah mendingin sepenuhnya dan deviasi sudut gun terhadap target kurang dari 8 derajat. Pendekatan ini merupakan keputusan greedy lokal yang memaksimalkan probabilitas peluru mengenai target pada setiap kesempatan tembak, daripada menembak sembarangan yang hanya membuang energi tanpa hasil.

2.2.3 Cara Menjalankan Bot

Langkah-langkah menjalankan bot:

4. Pastikan Robocode Tank Royale sudah terinstal.
5. Pastikan bot sudah terpasang local server
6. Masukan command `java -jar robocode-tankroyale-gui-0.30.0.jar` untuk membuka robocode royal nya.
7. Pilih folder bot kita di menu config.

8. Pilih Battle dan Start Battle untuk menjalankan server.
9. Di menu bot directory pilih folder bot lalu klik bot →.
10. Tunggu beberapa saat kemudian cek di Joined Bots.
11. Jika ingin battle pilih add dan pilih start battle

BAB 3: Aplikasi Strategi Greedy

3.1 Proses Mapping Persoalan Robocode Tank Royale

Pemetaan persoalan Robocode Tank Royale ke elemen-elemen algoritma Greedy dilakukan sebagai berikut:

3.1.1 Himpunan Kandidat

Himpunan kandidat merupakan seluruh kemungkinan aksi yang dapat dilakukan bot pada setiap tick game. Kandidat aksi bot mencakup:

1. Arah gerak badan: maju, mundur, atau diam
2. Arah rotasi badan: belok kiri atau belok kanan dengan berbagai sudut
3. Arah rotasi radar: putar kiri atau putar kanan
4. Arah rotasi gun: putar kiri atau putar kanan
5. Keputusan tembak: menembak dengan daya 0.1 hingga 3.0, atau tidak menembak
6. Pemilihan target: memilih salah satu dari seluruh musuh yang pernah terdeteksi radar sebagai target kunci. Pada setiap tick, bot harus memilih satu kombinasi aksi dari seluruh kandidat yang tersedia untuk dieksekusi.

3.1.2 Himpunan Solusi

Himpunan solusi adalah rangkaian aksi yang membentuk strategi lengkap bot sepanjang satu pertandingan. Solusi terbentuk secara inkremental setiap tick bot menambahkan satu set aksi ke dalam rangkaian solusi berdasarkan kondisi saat itu. Solusi dianggap lengkap ketika pertandingan berakhir, baik karena bot berhasil mengeliminasi seluruh musuh maupun karena bot sendiri dieliminasi. Rangkaian aksi yang membentuk solusi meliputi seluruh keputusan pergerakan, penguncian target, dan penembakan yang telah dieksekusi bot dari awal hingga akhir pertandingan.

3.1.3 Fungsi Solusi

Fungsi solusi mendefinisikan kapan suatu rangkaian aksi dianggap sebagai solusi yang valid. Dalam konteks bot ini, suatu rangkaian aksi dianggap solusi apabila bot masih bertahan hidup dengan energi di atas 0 dan setidaknya satu musuh berhasil dieliminasi. Solusi optimal tercapai ketika bot menjadi satu-satunya bot yang bertahan hidup di arena hingga pertandingan berakhir. Fungsi solusi dievaluasi secara kontinu setiap tick melalui pengecekan nilai EnemyCount dan Energy bot.

3.1.4 Fungsi Seleksi

Fungsi seleksi menentukan bagaimana bot memilih kandidat terbaik pada setiap langkah pengambilan keputusan. Bot mengimplementasikan fungsi seleksi melalui metode CalculateTargetScore yang mengevaluasi setiap musuh yang terdeteksi dan memilih musuh dengan skor tertinggi sebagai target. Fungsi seleksi ini bekerja secara greedy pada setiap event OnScannedBot, bot langsung membandingkan skor musuh yang baru terdeteksi dengan skor target yang sedang dikunci, dan segera mengalihkan kunci

apabila musuh baru memiliki skor yang lebih tinggi secara signifikan. Untuk keputusan tembak, fungsi seleksi memilih momen tembak yang paling optimal yaitu ketika gun sudah mendingin dan deviasi sudut gun terhadap target kurang dari 8 derajat.

3.1.5 Fungsi Kelayakan

Fungsi kelayakan memvalidasi apakah suatu kandidat aksi layak untuk dimasukkan ke dalam solusi. Sebuah kandidat aksi dinyatakan layak apabila memenuhi kondisi-kondisi berikut:

Kelayakan target: musuh yang dipilih sebagai target harus merupakan musuh yang terdeteksi dalam rentang waktu tidak lebih dari 12 tick terakhir, sehingga data posisinya masih relevan dan tidak kadaluarsa

Kelayakan tembak: bot hanya menembak apabila GunHeat bernilai 0, deviasi sudut gun kurang dari 8 derajat, dan energi bot masih mencukupi untuk menembak tanpa masuk ke kondisi kritis

Kelayakan daya tembak: daya tembak yang dipilih harus berada dalam rentang 0.1 hingga 3.0 dan disesuaikan dengan kondisi energi bot serta energi musuh agar tidak membuang energi secara sia-sia

Kelayakan gerak: aksi mundur dipilih hanya ketika jarak ke target kurang dari 180 unit untuk menghindari tabrakan, sementara aksi diam dipilih ketika jarak melebihi 500 unit agar bot tidak mengejar target secara tidak efisien.

3.1.6 Fungsi Objektif

Fungsi objektif mendefinisikan nilai yang dioptimalkan oleh algoritma greedy pada setiap langkah. Bot mengimplementasikan fungsi objektif melalui CalculateTargetScore dengan formula:

1. $\text{Skor} = (\text{distanceScore} \times 0.45) + (\text{killScore} \times 0.35) + (\text{aimScore} \times 0.15) - \text{speedPenalty} - \text{dangerPenalty}$
2. Di mana:
3. $\text{distanceScore} = \max(0, 1200 - \text{jarak}) / 1200 \times 45$, memaksimalkan prioritas musuh yang lebih dekat
4. $\text{killScore} = \max(0, 100 - \text{energiMusuh}) / 100 \times 35$, memaksimalkan prioritas musuh yang hampir mati
5. $\text{aimScore} = \max(0, 20 - |\text{gunBearing}|) / 20 \times 15$, memaksimalkan prioritas musuh yang sudah dalam jangkauan gun
6. $\text{speedPenalty} = \min(|\text{kecepatanMusuh}|, 8) \times 1.5$, mengurangi prioritas musuh yang bergerak cepat dan sulit ditembak
7. $\text{dangerPenalty} = 20$ apabila jarak musuh kurang dari 100 unit, mengurangi prioritas musuh yang terlalu dekat dan berbahaya

3.2 Eksplorasi 4 Alternatif Solusi Greedy

Berikut adalah tiga alternatif strategi greedy yang mungkin diterapkan dalam persoalan Robocode Tank Royale. Masing-masing alternatif memiliki heuristic yang berbeda:

3.2.1 Alternatif 1: Greedy by Maksimum Score

Alternatif pertama mengimplementasikan strategi greedy berbasis seleksi target dengan fungsi skor multikriteria. Heuristik utama bot ini adalah selalu mengunci dan menyerang musuh yang memiliki skor tertinggi pada saat itu, di mana skor dihitung dari kombinasi

empat faktor yaitu jarak, energi musuh, akurasi gun, dan kecepatan musuh. Bot beroperasi sepenuhnya melalui event OnScannedBot setiap kali radar mendeteksi musuh, bot langsung mengevaluasi apakah musuh tersebut lebih layak dijadikan target dibanding target yang sedang dikunci. Jika iya, kunci langsung dialihkan secara greedy tanpa pertimbangan lebih lanjut. Strategi ini efektif terhadap musuh yang bergerak terprediksi namun lemah menghadapi musuh dengan pola gerak tidak konvensional karena tidak memiliki mekanisme adaptasi strategi.

3.2.2 Alternatif 2: Greedy Corner Escape dengan Selective Fire

Alternatif kedua yang terdapat pada kode yang dikirim mengimplementasikan strategi greedy berbasis pemilihan posisi bertahan terbaik ketika menghadapi banyak musuh, dikombinasikan dengan mekanisme duel saat tinggal satu lawan. Heuristik utamanya terbagi dua: pada mode escape (musuh lebih dari satu), bot secara greedy memilih pojok arena yang memberikan total jarak terbesar dari semua musuh sekaligus mempertimbangkan apakah jalur ke pojok tersebut terhalang musuh, dengan skor pojok dihitung sebagai $\text{totalJarak} + \text{minJarak} \times 1.5 - \text{penaltiPath}$. Bot selalu memilih pojok dengan skor tertinggi setiap 20 tick secara adaptif. Pada mode duel (satu lawan), bot beralih ke strategi strafe perpendicular dengan kontrol jarak mundur jika terlalu dekat, diam jika terlalu jauh, dan zigzag di zona optimal. Penembakan pada mode escape dilakukan secara selektif hanya ketika musuh berada dalam jarak 400 unit dan energi bot mencukupi, mencerminkan keputusan greedy untuk menghemat energi sambil tetap memberikan tekanan pada musuh.

3.2.3 Alternatif 3: Greedy Survival dengan Daya Tembak Konservatif

Alternatif ketiga mengimplementasikan strategi greedy yang memprioritaskan kelangsungan hidup bot di atas agresivitas serangan. Heuristik utamanya adalah selalu menjaga jarak aman dari musuh sambil menembak dengan daya minimal yang masih efektif. Fungsi ChooseSafeFirePower mencerminkan keputusan greedy konservatif bot hanya menggunakan daya tembak maksimal 1.5 bahkan pada jarak dekat, jauh lebih rendah dibanding alternatif lain yang bisa mencapai 3.0, dan langsung menurunkan daya tembak ke 0.5 begitu energi bot di bawah 25. Pergerakan bot didasarkan pada tiga zona jarak: di bawah 220 unit bot langsung berbalik dan mundur, di atas 550 unit bot mengejar sambil strafe, dan di zona optimal bot bergerak zigzag tegak lurus terhadap musuh. Radar menggunakan multiplier 1.8 untuk penguncian yang lebih agresif dibanding alternatif lain. Strategi ini cocok untuk pertandingan panjang karena mengutamakan efisiensi energi, namun cenderung kurang agresif dalam menghabisi musuh karena daya tembak yang dibatasi secara greedy berdasarkan kondisi energi saat itu.

3.3 Analisis Efisiensi dan Efektivitas Solusi

Perbandingan Ketiga alternatif solusi greedy dilakukan berdasarkan dua dimensi utama yaitu efisiensi komputasi dan efektivitas strategi dalam pertandingan.

Dari sisi efisiensi komputasi, Alternatif 1 (Greedy by Maximum Score) memiliki kompleksitas komputasi tertinggi karena harus menghitung fungsi skor multikriteria setiap kali event scan dipanggil. Namun karena perhitungan dilakukan hanya saat radar mendeteksi musuh dan bukan setiap tick, overhead komputasinya tetap dapat ditoleransi.

Alternatif 3 (Greedy Survival) memiliki kompleksitas paling rendah karena fungsi ChooseSafeFirePower hanya melibatkan perbandingan nilai sederhana tanpa perhitungan multikriteria. Alternatif 2 (Greedy Corner Escape) memiliki kompleksitas menengah namun dengan overhead tambahan pada fungsi PickBestCorner yang harus mengevaluasi empat pojok sekaligus memverifikasi apakah jalur ke masing-masing pojok terhalang musuh setiap 20 tick.

3.4 Strategi Greedy yang Dipilih

Dari ketiga alternatif di atas, strategi yang dipilih untuk diimplementasikan adalah:

Alternatif 1: Greedy by Maximum Score

Alasan dan pertimbangan pemilihan strategi:

strategi Greedy by Maximum Score dipilih karena memiliki fungsi objektif yang paling komprehensif dan terukur dibanding alternatif lainnya. Dengan menggabungkan empat kriteria sekaligus dalam satu nilai skor, bot mampu membuat keputusan pemilihan target yang lebih cerdas dan kontekstual tidak hanya berdasarkan jarak atau energi saja, melainkan mempertimbangkan seluruh faktor relevan secara bersamaan dalam setiap keputusan greedy.

BAB 4: Implementasi dan Pengujian

4.1 Implementasi 4 Alternatif Solusi (Pseudocode)

Berikut adalah pseudocode untuk ketiga alternatif solusi greedy yang telah diidentifikasi. Pseudocode dibuat cukup detail sehingga dapat dimengerti dengan mudah.

4.1.1 Pseudocode Alternatif 1

Greedy by Maximum Score

VARIABEL:

```
targetId      ← -1
targetScore   ← -1
targetLastSeen ← -1000
moveDir       ← 1
radarDir      ← 1
lastScanTurn  ← -1000
```

FUNGSI CalculateTargetScore(jarak, energiMusuh, gunBearing, kecepatanMusuh):

```
distanceScore ← max(0, 1200 - jarak) / 1200 * 45
killScore     ← max(0, 100 - energiMusuh) / 100 * 35
aimScore      ← max(0, 20 - |gunBearing|) / 20 * 15
speedPenalty  ← min(|kecepatanMusuh|, 8) * 1.5
```

```

    dangerPenalty ← JIKA jarak < 100 MAKA 20 SELAINNYA 0
    RETURN distanceScore + killScore + aimScore -
    speedPenalty - dangerPenalty

```

FUNGSI ChooseFirePower(jarak, energiMusuh):

```

    JIKA jarak < 150          : daya ← 3.0
    SELAIN JIKA jarak < 350   : daya ← 2.2
    SELAIN JIKA jarak < 650   : daya ← 1.4
    SELAINNYA                 : daya ← 0.8

    JIKA energiMusuh < 12:
        daya ← min(daya, energiMusuh / 3.0 + 0.1)
    JIKA energiBot < 20:
        daya ← min(daya, 1.0)

    RETURN clamp(daya, 0.1, 3.0)

```

PROSEDUR Run():

```

    atur AdjustGunForBodyTurn ← true
    atur AdjustRadarForBodyTurn ← true
    atur AdjustRadarForGunTurn ← true

```

SELAMA bot berjalan:

```

    putar badan kiri 20 derajat
    maju 120 * moveDir
    putar radar kiri 45 * radarDir derajat
    eksekusi tick

```

PROSEDUR OnScannedBot(musuh):

```

    lastScanTurn ← tick sekarang
    jarak        ← jarak ke musuh
    gunBearing    ← sudut gun ke musuh
    bodyBearing   ← sudut badan ke musuh
    skor          ← CalculateTargetScore(jarak, musuh.energi,
    gunBearing, musuh.kecepatan)

    targetKadaluarsa ← (tickSekarang - targetLastSeen > 12)
    targetSama        ← (musuh.id == targetId)

```

```
targetLebihBaik ← (skor > targetScore + 8)

JIKA targetId == -1 ATAU targetSama ATAU
targetKadaluarsa ATAU targetLebihBaik:
    targetId      ← musuh.id
    targetScore   ← skor
    targetLastSeen ← tickSekarang
SELAINNYA:
    lakukan rescan
RETURN

// Arahkan badan strafe tegak lurus ke target
putar badan kiri (bodyBearing + 90 - 20 * moveDir)

// Arahkan gun ke target
putar gun kiri gunBearing

// Kunci radar ke target
putar radar kiri RadarBearingTo(musuh) * 1.5

// Kontrol jarak
JIKA jarak < 180:
    moveDir ← -moveDir
    mundur 100
SELAIN JIKA jarak > 500:
    maju 100
SELAINNYA:
    maju 100 * moveDir

// Tembak jika kondisi optimal
JIKA GunHeat == 0 DAN |gunBearing| < 8:
    tembak ChooseFirePower(jarak, musuh.energi)

lakukan rescan

PROSEDUR OnHitByBullet(peluru):
    moveDir ← -moveDir
    arahPeluru ← hitung bearing arah datang peluru
```

```

    putar badan kiri (90 - arahPeluru)
    maju 140 * moveDir
    putar radar kiri 45 * radarDir

```

PROSEDUR OnHitBot(musuh) :

```

    moveDir      ← -moveDir
    jarak        ← jarak ke musuh
    gunBearing   ← sudut gun ke musuh
    putar gun kiri gunBearing
    JIKA musuh.energi < 16 DAN GunHeat == 0:
        tembak ChooseFirePower(jarak, musuh.energi)
    mundur 100
    putar badan kiri 60

```

PROSEDUR OnHitWall() :

```

    moveDir      ← -moveDir
    radarDir     ← -radarDir
    mundur 120
    putar badan kiri 70

```

4.2 Penjelasan Struktur, Fungsi, dan Prosedur

Berikut adalah penjelasan struktur data, fungsi, dan prosedur yang digunakan pada bot dengan solusi greedy yang dipilih:

4.2.1 Struktur Data

Bot menggunakan beberapa variabel state untuk menyimpan informasi yang diperlukan dalam pengambilan keputusan greedy:

Variabel	Tipe Data	Fungsi
targetId	int	Menyimpan ID musuh yang sedang dikunci, bernilai -1 jika belum ada target
targetScore	double	Menyimpan skor greedy target yang sedang dikunci sebagai pembanding
targetLastSeen	int	Menyimpan tick terakhir target terdeteksi untuk mendeteksi target kadaluarsa

moveDir	int	Menyimpan arah gerak aktif (+1 maju, -1 mundur)
radarDir	int	Menyimpan arah rotasi radar (+1 kiri, -1 kanan)
lastScanTurn	int	Menyimpan tick terakhir radar melakukan scan

4.2.2 Fungsi-Fungsi Utama

CalculateTargetScore(distance, enemyEnergy, gunBearing, enemySpeed)

Fungsi objektif greedy yang menghitung skor prioritas setiap musuh. Mengembalikan nilai numerik yang merepresentasikan seberapa menguntungkan musuh tersebut untuk dijadikan target saat ini. Skor yang lebih tinggi berarti musuh tersebut lebih diprioritaskan untuk dieliminasi. ChooseFirePower(distance, energyEnemy) Fungsi seleksi daya tembak yang menentukan besar daya tembak optimal berdasarkan jarak ke target dan kondisi energi kedua pihak. Mengembalikan nilai daya tembak dalam rentang 0.1 hingga 3.0. OnScannedBot(ScannedBotEvent e) Fungsi utama yang menangani seluruh logika duel bot. Dipanggil setiap kali radar mendeteksi musuh. Di dalam fungsi ini terjadi evaluasi target, penguncian radar, manuver strafe, pengarahan gun, dan keputusan tembak secara terintegrasi dalam satu alur greedy.

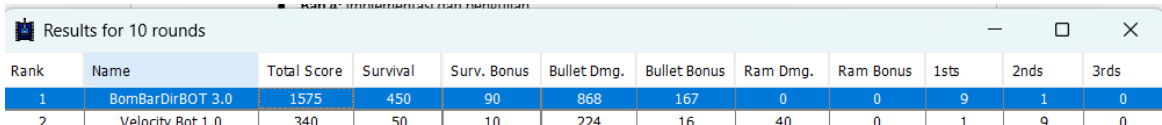
4.2.3 Prosedur Utama

Run() Prosedur utama yang berjalan dalam loop tak terhingga selama bot hidup. Prosedur ini mengatur pergerakan dasar bot yaitu rotasi badan, gerak maju, dan sweep radar secara kontinu. Prosedur ini berfungsi sebagai fallback movement ketika tidak ada event scan yang aktif. OnHitByBullet(HitByBulletEvent e) Prosedur respons terhadap event terkena peluru. Bot membalik arah gerak dan bermanuver tegak lurus dari arah datangnya peluru untuk keluar dari lintasan tembak musuh. OnHitBot(HitBotEvent e) Prosedur respons terhadap event tabrakan dengan bot lain. Bot membalik arah gerak, mundur, dan memanfaatkan momen jarak dekat untuk menembak apabila musuh dalam kondisi energi rendah. OnHitWall(HitWallEvent e) Prosedur respons terhadap event menabrak dinding. Bot membalik arah gerak dan arah radar, kemudian mundur dan berbelok untuk kembali ke area arena yang lebih terbuka.

4.3 Pengujian

Pengujian dilakukan minimal sebanyak 3 kali berupa pertandingan antara bot utama (yang dipilih) dengan bot asisten (format 1 vs 1). Pengujian mencakup semua kejadian yang unik.

4.3.1 Pengujian 1 - Lawan Velocity Bot 1.0



Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	BomBarDirBOT 3.0	1575	450	90	868	167	0	0	9	1	0
2	Velocity Bot 1.0	340	50	10	224	16	40	0	1	9	0

Hasil: BomBarDirBOT menang 9 dari 10 ronde.

Analisis: BomBarDirBOT berhasil mendominasi pertandingan dengan memanfaatkan sistem seleksi target berbasis skor yang mengunci Velocity Bot sebagai prioritas utama. Bullet Damage yang tinggi (868) menunjukkan bahwa mekanisme tembak adaptif bekerja efektif dalam kondisi jarak menengah. Kekalahan pada 1 ronde terjadi karena Velocity Bot sesekali melakukan manuver menjauh secara agresif sehingga jarak antara kedua bot melebihi 500 unit. Pada kondisi ini BomBarDirBOT berhenti bergerak dan hanya menembak dari jarak jauh, namun Velocity Bot yang terus bergerak zigzag menyebabkan akurasi tembakan menurun dan bot kesulitan mengenai sasaran hingga akhirnya kalah di ronde tersebut.

4.3.2 Pengujian 2

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	BomBarDirBOT 3.0	1767	500	100	973	194	0	0	10	0	0
2	Corners 1.0	80	0	0	76	0	4	0	0	10	0

Hasil: BomBarDirBOT menang sempurna 10 dari 10 ronde.

Analisis: BomBarDirBOT berhasil memenangkan seluruh ronde tanpa sekalipun kalah. Corners 1.0 yang mengandalkan strategi berlindung di sudut arena menjadi sasaran yang relatif mudah karena posisinya yang cenderung statis dan mudah diprediksi. Sistem penguncian radar BomBarDirBOT berhasil mempertahankan kunci pada Corners 1.0 sepanjang pertandingan, menghasilkan Bullet Damage tertinggi di antara ketiga pengujian yaitu 973 dengan Bullet Bonus 194 yang menunjukkan tingkat akurasi tembakan yang sangat baik. Corners 1.0 hanya mampu memberikan Ram Damage sebesar 4, menandakan bot tersebut tidak mampu mendekat secara efektif untuk memberikan perlawanan berarti.

4.3.3 Pengujian 3

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Spin Bot 1.0	1314	450	90	672	102	0	0	9	1	0
2	BomBarDirBOT 3.0	558	50	10	479	18	0	0	1	9	0

Hasil: BomBarDirBOT kalah 1 dari 10 ronde, menang hanya 1 ronde.

Analisis: Pengujian ini mengungkap kelemahan utama BomBarDirBOT ketika menghadapi musuh dengan pola gerak berputar konstan. Spin Bot 1.0 yang terus berputar pada porosnya menyebabkan sudut gun BomBarDirBOT tidak pernah cukup stabil untuk memenuhi syarat tembak yaitu deviasi sudut kurang dari 8 derajat, sehingga sebagian besar kesempatan tembak terlewat. Selain itu, pola gerak melingkar Spin Bot menyebabkan jarak antara kedua bot berfluktuasi secara tidak terprediksi, membuat sistem kontrol jarak BomBarDirBOT kesulitan menentukan momen yang tepat untuk maju atau mundur. Bullet Bonus yang sangat rendah (18) dibandingkan pengujian lain mencerminkan bahwa sangat sedikit peluru yang berhasil mengenai sasaran secara akurat selama pertandingan berlangsung.

4.4 Analisis Hasil Pengujian

Berdasarkan hasil ketiga pengujian, strategi greedy yang diimplementasikan pada BomBarDirBOT menunjukkan performa yang optimal pada kondisi tertentu namun memiliki keterbatasan pada kondisi lainnya.

Strategi greedy terbukti bekerja optimal pada Pengujian 2 melawan Corners 1.0, di mana bot berhasil memenangkan seluruh 10 ronde tanpa kekalahan. Kondisi ini terjadi karena musuh bergerak secara statis dan terprediksi, sehingga fungsi seleksi target berbasis skor dapat mengevaluasi dan mengunci target secara konsisten setiap tick. Keputusan greedy lokal pada setiap langkah memilih target dengan skor tertinggi, menentukan daya tembak optimal, dan menembak hanya saat akurasi terjamin menghasilkan keputusan yang sekaligus merupakan keputusan global optimal karena kondisi musuh tidak berubah secara drastis.

Strategi greedy bekerja cukup optimal pada Pengujian 1 melawan Velocity Bot 1.0 dengan kemenangan 9 dari 10 ronde. Namun satu kekalahan mengungkap keterbatasan sifat greedy yang hanya mempertimbangkan kondisi saat ini tanpa perencanaan jangka panjang. Ketika musuh menjauh melampaui 500 unit, bot secara greedy memutuskan untuk diam dan menembak dari jauh karena itulah keputusan yang tampak optimal saat itu. Namun keputusan ini tidak mempertimbangkan bahwa musuh yang terus bergerak zigzag akan semakin sulit ditembak dari jarak jauh, sehingga seharusnya bot tetap mengejar untuk mempertahankan jarak optimal.

Strategi greedy tidak optimal pada Pengujian 3 melawan Spin Bot 1.0 yang menghasilkan kekalahan 9 dari 10 ronde. Ini merupakan contoh klasik kegagalan algoritma greedy akibat sifat shortsighted, bot hanya mengevaluasi kondisi saat ini tanpa mengantisipasi pola gerak musuh ke depan. Fungsi seleksi tembak yang mensyaratkan deviasi sudut kurang dari 8 derajat menjadi kontraproduktif menghadapi musuh yang berputar konstan, karena kondisi ideal tersebut hampir tidak pernah tercapai. Algoritma greedy tidak mampu beradaptasi dengan mengubah strategi secara fundamental ketika menghadapi pola musuh yang benar-benar berbeda dari asumsi dasarnya.

Secara keseluruhan, strategi greedy pada BomBarDirBOT berhasil optimal pada kondisi musuh yang bergerak terprediksi dan berada dalam jarak menengah, namun tidak optimal ketika menghadapi musuh dengan pola gerak yang tidak konvensional atau ketika kondisi jarak berubah drastis secara terus-menerus.

BAB 5: Kesimpulan dan Saran

5.1 Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

Pertama, algoritma greedy berhasil diimplementasikan pada BomBarDirBOT melalui empat mekanisme utama yaitu seleksi target berbasis skor, pemilihan daya tembak adaptif, manuver strafe perpendicular, dan penguncian radar dinamis. Setiap mekanisme mengambil keputusan lokal optimal pada setiap langkah tanpa mempertimbangkan konsekuensi jangka panjang, yang merupakan karakteristik utama algoritma greedy.

Kedua, pendekatan greedy terbukti efektif dan menghasilkan solusi yang mendekati optimal ketika menghadapi musuh dengan pola gerak yang statis atau terprediksi. Hal ini dibuktikan dengan kemenangan sempurna 10-0 melawan Corners 1.0 dan dominasi 9-1 melawan Velocity Bot 1.0.

Ketiga, algoritma greedy memiliki keterbatasan mendasar berupa ketidakmampuan beradaptasi terhadap pola musuh yang tidak konvensional. Kekalahan telak 1-9 melawan Spin Bot 1.0 membuktikan bahwa keputusan greedy yang optimal secara lokal tidak selalu menghasilkan solusi yang optimal secara global, terutama ketika asumsi dasar strategi tidak terpenuhi di lapangan.

Ketiga, sifat shortsighted dari algoritma greedy menjadi kelemahan nyata dalam konteks Robocode Tank Royale yang dinamis, di mana kondisi arena dapat berubah drastis dalam beberapa tick sehingga keputusan yang tampak optimal saat ini bisa menjadi keputusan yang merugikan beberapa tick kemudian.

5.2 Saran

Berdasarkan hasil analisis, beberapa saran untuk pengembangan BomBarDirBOT ke depannya adalah sebagai berikut:

Pertama, untuk mengatasi kelemahan menghadapi musuh berputar seperti Spin Bot, dapat ditambahkan mekanisme deteksi pola gerak musuh. Apabila musuh terdeteksi berputar konstan, bot dapat mengadopsi strategi tembak berbasis prediksi posisi masa depan alih-alih menunggu kondisi akurasi statis terpenuhi, misalnya dengan menghitung titik intersepsi antara peluru dan lintasan gerak melingkar musuh.

Kedua, kondisi diam saat jarak melebihi 500 unit sebaiknya diganti dengan mekanisme pengejaran aktif yang tetap mempertahankan jarak optimal. Bot seharusnya mempertimbangkan tidak hanya kondisi saat ini tetapi juga tren perubahan jarak, apabila musuh terus menjauh, bot harus mengejar alih-alih menembak dari jarak yang semakin tidak efektif.

Ketiga, threshold deviasi sudut untuk menembak sebesar 8 derajat dapat dibuat dinamis berdasarkan kecepatan gerak musuh. Musuh yang bergerak sangat cepat atau berputar memerlukan threshold yang lebih longgar dengan kompensasi prediksi posisi, sementara musuh yang diam dapat ditembak dengan threshold yang lebih ketat untuk memaksimalkan akurasi.

Ketiga, untuk pengembangan jangka panjang, dapat dipertimbangkan kombinasi algoritma greedy dengan pendekatan yang lebih adaptif seperti pengenalan pola atau machine learning sederhana, sehingga bot dapat mengidentifikasi strategi musuh dan menyesuaikan fungsi objektif greedy secara dinamis sesuai dengan karakteristik lawan yang dihadapi.

Lampiran

Repository GitHub

https://github.com/ikhromazhar/Tubes_BomBarDir.git

Video Demonstrasi (Jika Ada)

<https://youtu.be/nf2yW80aRHg?si=7sPiL61Av1izakqa>

Daftar Pustaka
